

Stima della cardinalità nei flussi di dati distribuiti: merge e compatibilità semantica degli algoritmi di sketch

Seminario di tesi magistrale

Daniele Ferroli

Università degli Studi di Udine

A.A. 2024–2025

- 1 Motivazione e domanda di ricerca
- 2 Sketch count-distinct
- 3 Merge e compatibilità semantica
- 4 Infrastruttura sperimentale
- 5 Risultati sugli algoritmi
- 6 Risultati sul merge distribuito
- 7 Conclusioni

Motivazione e domanda di ricerca

- Il problema riguarda la stima degli **elementi distinti** che compaiono in un flusso di dati continuo [1, 2].
- Il conteggio esatto diventa rapidamente costoso in memoria e tempo quando il numero di elementi distinti cresce e la risposta deve essere aggiornata con bassa latenza.
- Le applicazioni tipiche includono telemetria, monitoraggio di rete, osservabilità, analisi di log e sistemi distribuiti.
- Nei contesti distribuiti non basta una buona stima locale: serve anche poter **combinare correttamente** i risultati prodotti su nodi diversi.

Punto chiave

Il lavoro non riguarda solo quanto bene gli algoritmi stimino gli elementi distinti, ma come si comporta il **merge** in situazioni eterogenee o non ottimali.

Dal conteggio esatto allo sketch

Conteggio esatto

$$\begin{aligned} S_t &= \langle x_1, \dots, x_t \rangle, & V_t &= \{x_j \mid 1 \leq j \leq t\} \\ V_i &\leftarrow V_{i-1} \cup \{x_i\}, & 1 \leq i \leq t, \\ |V_t| &= F_0(t) \end{aligned}$$

Memoria $\Theta(|V_t|) = \Theta(F_0(t))$: cresce con il numero di distinti osservati.

Sketch

$$\begin{aligned} S_t &= \langle x_1, \dots, x_t \rangle, & M_0 &= \text{sketch vuoto} \\ M_i &\leftarrow \text{Update}(M_{i-1}, x_i), & 1 \leq i \leq t, \\ \widehat{F}_0(t) &\leftarrow \text{Query}(M_t) \end{aligned}$$

Si mantiene uno stato compatto e si restituisce una stima di $F_0(t)$ [3].

Complessità dello sketch

Per una $(1 \pm \varepsilon)$ -stima con probabilità $1 - \delta$ [4]:

$$\text{space}(M_t) = O(\varepsilon^{-2} \log \log n + \log n), \quad n = |\mathcal{U}|.$$

Update per elemento: $O(1)$ operazioni sullo stato. Il compromesso resta tra memoria, costo di aggiornamento e accuratezza; nel caso distribuito si aggiunge il merge degli sketch locali.

$$S_1 \longrightarrow K_1 \quad S_2 \longrightarrow K_2$$

Per un algoritmo fissato $\mathcal{A}$, $\oplus_{\mathcal{A}}$ denota il merge su sketch compatibili [5]; K_{serial} è lo sketch ottenuto processando $S_1 \parallel S_2$, con $\parallel$ concatenazione dei flussi.

$$\text{Query}(K_1 \oplus_{\mathcal{A}} K_2) = \text{Query}(K_{\text{serial}})$$

- Ogni nodo mantiene uno sketch locale della propria partizione del flusso o della propria partizione temporale.
- In un sistema possono cambiare algoritmo, funzione di hash, seed e precisione dello sketch.
- Il punto è capire quando il merge coincide ancora col caso seriale.

Domanda di ricerca

Quando il merge è semanticamente corretto, quando è recuperabile e quando invece deve essere rifiutato?

Risultato principale

Negli sketch count-distinct distribuiti, hash, seed e parametri non sono dettagli di implementazione: fanno parte della **semantica** dello sketch e quindi della correttezza del merge.

Contributi:

- Implementazione degli algoritmi Probabilistic Counting, LogLog, HyperLogLog e HyperLogLog++.
- Framework sperimentale che separa algoritmi, hashing, dataset, metriche e risultati.
- Classificazione operativa dei casi di merge in *valid*, *recoverable* e *invalid*.
- Valutazione empirica di casi eterogenei cercati, con HLL++ come riferimento principale.

Non viene proposto un **nuovo stimatore teorico**: il contributo è uno **studio sistematico** del comportamento degli sketch e del merge su un dataset sintetico con proprietà controllate.

Sketch count-distinct

$$S = \langle x_1, x_2, \dots, x_s \rangle, \quad x_i \in \mathcal{U}$$

$$f(a) = |\{i \mid x_i = a\}| \quad F_0 = |\{a \in \mathcal{U} \mid f(a) > 0\}|$$

- S è un flusso ordinato di elementi osservati.
- $f(a)$ conta quante volte compare la chiave a .
- F_0 è il numero di elementi distinti nel flusso, cioè il momento di frequenza di ordine zero [6].
- Nel lavoro si considera il modello *insertion-only*.

Definizione

Un algoritmo per flussi di dati elabora $S = \langle x_1, \dots, x_s \rangle$ in un solo passaggio, mantiene uno stato interno M_i di dimensione limitata e aggiorna tale stato dopo ogni elemento osservato. In qualunque momento può restituire una risposta interrogando lo stato.

$$S_i = \langle x_1, \dots, x_i \rangle,$$

$$M_0 = \text{stato vuoto}$$

$$M_i \leftarrow \text{Update}(M_{i-1}, x_i),$$

$$\widehat{F}_0(S_i) \leftarrow \text{Query}(M_i)$$

- M_i deve restare piccolo rispetto al prefisso processato.
- Update deve essere veloce, perché avviene per ogni elemento.
- La qualità si misura confrontando $\widehat{F}_0(S_i)$ con $F_0(S_i)$.

Stima

Una stima (ε, δ) -approssimata richiede

$$\Pr\left(|\widehat{F}_0(S_i) - F_0(S_i)| \leq \varepsilon F_0(S_i)\right) \geq 1 - \delta.$$

Famiglie implementate e valutate

Nel lavoro sperimentale vengono considerati algoritmi probabilistici per la stima di F_0 , tutti basati sull'uso di funzioni di hash e su uno stato compatto aggiornato elemento per elemento.

- **Probabilistic Counting**: mantiene una bitmap e stima la cardinalità a partire dalla posizione dei bit osservati [7].
- **LogLog**: usa registri che memorizzano massimi di rango prodotti dall'hash [8].
- **HyperLogLog**: mantiene la struttura a registri, ma cambia lo stimatore usando la media armonica [9].
- **HyperLogLog++**: variante pratica di HyperLogLog con rappresentazione sparsa/densa, correzioni del bias e gestione dei regimi piccoli [10].

Struttura della famiglia di algoritmi LogLog

Precisione k : si usano $m = 2^k$ registri $M[0], \dots, M[m-1]$.

Con un hash di L^* bit:

$$h(x) = \underbrace{j}_{k \text{ bit}} \parallel \underbrace{w}_{L-k \text{ bit}}, \quad r = \text{bin}(j), \quad \rho(w) = 1 + \text{zeri iniziali}^{**} \text{ in } w$$

* = 32 bit in genere

** = leggendo i bit da sinistra a destra

Aggiornamento

$$M[r] \leftarrow \max\{M[r], \rho(w)\}$$

Ogni registro memorizza il massimo rango osservato sugli elementi assegnati a quel registro.

Stima LogLog

$$A = \frac{1}{m} \sum_{r=0}^{m-1} M[r], \quad \widehat{F}_0 = \alpha_m \cdot m \cdot 2^A$$

LogLog usa la media dei ranghi e risale alla cardinalità su scala esponenziale.

La famiglia condivide la struttura a registri. Nei miglioramenti successivi viene modificata la funzione di stima e vengono applicate correzioni sulla stima in maniera empirica.

```
Choose the precision  $k$  and set  $m = 2^k$   
Initialize the registers  $M[0], \dots, M[m-1] \leftarrow 0$   
for each element  $x$  in the stream do  
     $y \leftarrow h(x)$   
     $j \leftarrow \text{Prefix}_k(y)$   
     $w \leftarrow \text{Suffix}_{L-k}(y)$   
     $M[j] \leftarrow \max\{M[j], \rho(w)\}$   
end for  
 $A \leftarrow \frac{1}{m} \sum_{j=0}^{m-1} M[j]$   
return  $\widehat{F}_0 \leftarrow \alpha_m \cdot m \cdot 2^A$ 
```

LogLog: esempio operativo

Consideriamo $k = 2$, quindi $m = 4$ registri, e hash su $L = 8$ bit. I primi due bit selezionano il registro j ; i restanti sei bit formano il suffisso w , da cui si calcola $\rho(w)$.

Passo	x	$y = h(x)$	j	w	$\rho(w)$	Stato registri	$\widehat{F}_0$
1	a	10010100_2	2	010100_2	2	$(0, 0, 2, 0)$	≈ 2.25
2	b	10111000_2	2	111000_2	1	$(0, 0, 2, 0)$	≈ 2.25
3	c	01001100_2	1	001100_2	3	$(0, 3, 2, 0)$	≈ 3.78
4	d	11001000_2	3	001000_2	3	$(0, 3, 2, 3)$	≈ 6.352
5	e	10000100_2	2	000100_2	4	$(0, 3, 4, 3)$	≈ 8.98

Merge e compatibilità semantica

Il merge corretto richiede

$$\text{Query}(K_1 \oplus_{\mathcal{A}} K_2) = \text{Query}(K_{\text{serial}})$$

Famiglia a registri

LogLog, HLL e HLL++.

$$\forall j \in \{0, \dots, m-1\}, \quad M[j] = \max\{M_1[j], M_2[j]\}$$

Merge componente per componente [11].

L'operazione $\oplus_{\mathcal{A}}$ è corretta solo con **algoritmo**, **hash/seed** e **parametri strutturali** invariati.

Classificazione dei casi di merge

Il controllo di compatibilità produce tre esiti: merge diretto, merge dopo normalizzazione, oppure rifiuto del merge.

valid

Merge diretto.
Stesso algoritmo, stessa hash,
stesso seed, stessi parametri.

recoverable

Merge corretto dopo
normalizzazione.
HLL++ con precisioni diverse e
stessa semantica hash.

invalid

Merge da rifiutare.
Funzioni di hash o seed diversi.

Strategie utilizzate

valid $\rightarrow$ direct, recoverable $\rightarrow$ reduce_then_merge, invalid $\rightarrow$ reject.
unsafe_naive_merge è usato solo come controllo negativo.

Infrastruttura sperimentale

Infrastruttura sperimentale

Il framework separa algoritmi, hashing, dataset e valutazione, così da isolare gli effetti del merge.

Algoritmi

API comune per process, count, merge, reset.
Implementazioni: PC, LogLog, HLL, HLL++ e algoritmo naive.

Hash e dataset

Funzioni di hash separate dal livello dello sketch.
Seed, parametri e metadati sono tracciati esplicitamente.

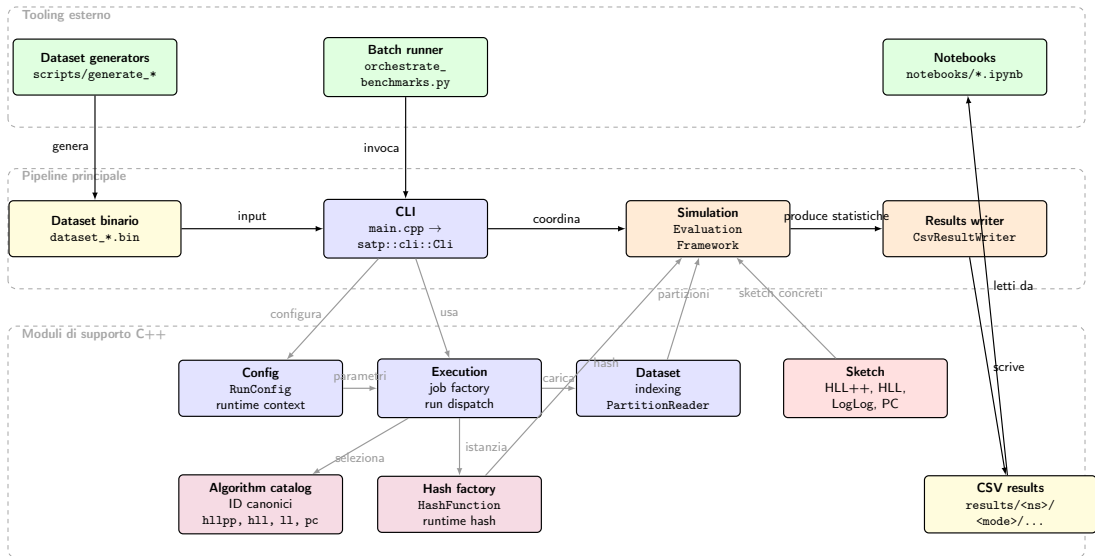
Valutazione

Modalità streaming, merge omogeneo e merge eterogeneo.
Metriche e risultati sono serializzati in CSV riproducibili.

Scelta progettuale

Il framework rende osservabile la compatibilità semantica del merge sotto condizioni riproducibili.

Architettura del sistema



Dataset sintetico: protocollo di generazione

Parametri

Per ogni dataset: $n = 10^7$ elementi per partizione, $p = 50$ partizioni, d elementi distinti, seed globale = 21041998,

$$\rho = \frac{n}{d} \in \{100, 50, 20, 10, 5, 2, 1\}, \quad n \bmod d = 0.$$

Costruzione di una partizione

- 1 seed locale deterministico dal seed globale e dall'indice di partizione;
- 2 estrazione di d interi distinti;
- 3 una nuova prima occorrenza all'inizio di ogni blocco di lunghezza ρ ;
- 4 completamento del blocco campionando tra gli identificatori già visti.

Proprietà indotta

Ogni blocco introduce esattamente un nuovo elemento distinto. Quindi, al bordo del j -esimo blocco,

$$F_0(j \cdot \rho) = j.$$

Il protocollo separa in modo controllato la crescita di $F_0(t)$ dal numero medio di ripetizioni.

Dataset sintetico: esempio di partizione

Esempio piccolo

Supponiamo $n = 12$, $d = 4$ e quindi $\rho = n/d = 3$. La partizione è divisa in quattro blocchi di lunghezza 3: all'inizio di ogni blocco compare una nuova prima occorrenza.

t	1	2	3	4	5	6	7	8	9	10	11	12
blocco	1			2			3			4		
x_t	a	a	a	b	a	b	c	a	c	d	b	c
b_t	1	0	0	1	0	0	1	0	0	1	0	0
$F_0(t)$	1	1	1	2	2	2	3	3	3	4	4	4

Cosa viene memorizzato

Il dataset contiene la sequenza dei valori x_t e un bitset di verità: $b_t = 1$ esattamente nelle posizioni in cui compare un nuovo distinto.

Verità di prefisso

La cardinalità reale del prefisso si ricostruisce sommando il bitset:

$$F_0(t) = \sum_{i=1}^t b_i.$$

Risultati sugli algoritmi

Protocollo sperimentale: organizzazione

Il protocollo segue la validazione del contesto, gli esperimenti sugli algoritmi e il merge distribuito.

Streaming

Si osserva la stima lungo i prefissi del flusso, confrontando $\widehat{F}_0(t)$ con la cardinalità reale $F_0(t)$.

Precisione

Si mantiene fisso il contesto di esecuzione e si varia il parametro interno di ciascun algoritmo.

Merge

Si confrontano merge omogeneo e casi eterogenei, distinguendo tra `valid`, `invalid` e `recoverable`.

Metriche principali

Streaming: media della stima, dispersione, bias ed errore relativo medio.

Merge: scostamento tra stima da merge e riferimento seriale.

Struttura comune per gli esperimenti in modalità streaming

Configurazione

Dataset `prefix_constant_rho` con $n = 10^7$, $p = 50$, `seed = 21041998` e

$$\rho \in \{1, 10, 100\}.$$

Griglie: HLL++ con $k \in \{8, 10, 12, 14, 16, 18\}$; HLL e LogLog con $k \in \{4, 6, 8, 10, 12, 14, 16\}$; Probabilistic Counting con $L \in \{4, 8, 12, 16, 20, 24, 28, 31\}$.

Struttura dell'analisi

Non si osservano tutti i 10^7 elementi del flusso, ma 200 punti di controllo fissati in anticipo.

Nel caso $n = 10^7$, il pianificatore:

- distribuisce circa un terzo dei checkpoint in modo lineare nel primo 1% del flusso;
- distribuisce i restanti checkpoint fino a n con passo crescente;
- include sempre $t = 1$ e $t = n$.

Lettura intuitiva

In pratica si fanno 200 “istantanee” del flusso: fitte all’inizio, dove la stima cambia più rapidamente, e poi via via più rade.

A ciascun checkpoint si confrontano $F_0(t)$ e $\widehat{F}_0(t)$; sulle 50 partizioni si aggregano poi media e dispersione.

Esperimenti sugli algoritmi: lettura dei risultati

Idea

L'analisi della robustezza separa due effetti: crescita della cardinalità reale del prefisso e rapporto medio di ripetizione.

Caso A

Si osserva $\widehat{F}_0(t)$ in funzione di $F_0(t)$ a ρ fissato.
Questa vista mostra come cambiano bias e dispersione al crescere degli distinti reali nel prefisso.

Caso B

Si fissano sette valori target

$$F_0^* \in \{10^3, 2 \cdot 10^3, 5 \cdot 10^3, 10^4, 2 \cdot 10^4, 5 \cdot 10^4, 10^5\}$$

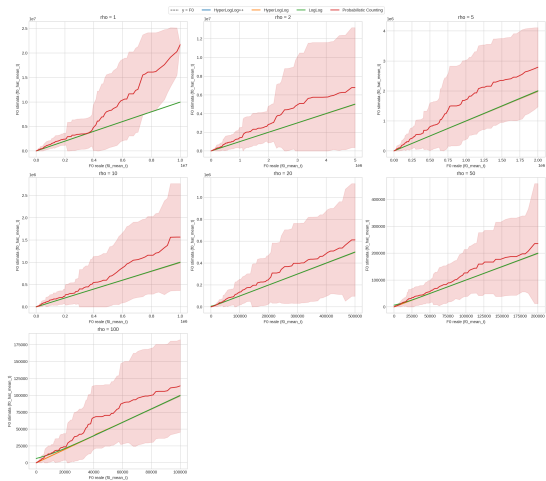
e si legge la stima in funzione di

$$\rho_t = \frac{t}{F_0(t)}.$$

Il confronto finale usa l'errore relativo medio ai punti finali osservati.

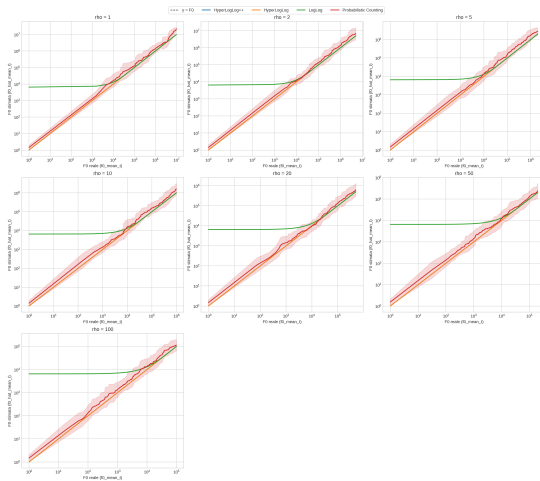
Caso A: stima rispetto alla cardinalità reale

Type A - prebq_constant_rho (n=1000000, seed=21042998, scala lineare)



Scala lineare.

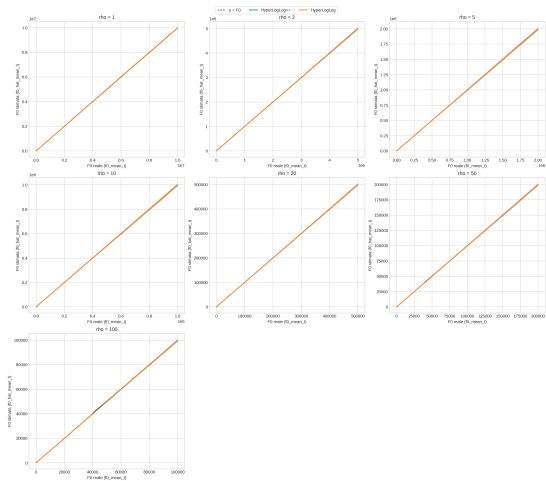
Type A - prebq_constant_rho (n=1000000, seed=21042998, scala log log)



Scala logaritmica.

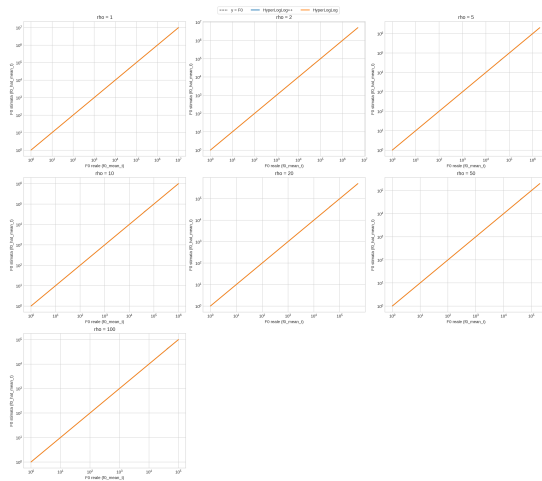
Caso A: confronto HyperLogLog e HyperLogLog++

Type A (solo HLL/HLL++) - prefix_constant_rho (n=1000000, seed=23041993, scale linear)



Scala lineare.

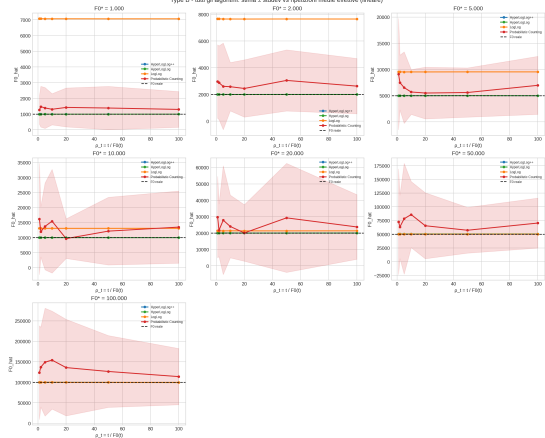
Type A (solo HLL/HLL++) - prefix_constant_rho (n=1000000, seed=23041993, scale log-log)



Scala logaritmica.

Caso B: rapporto tra elementi processati e distinti

Type B - tutti gli algoritmi: stima $\pm$ stdev vs (ripetizioni medie effettive (lineare)



Scala lineare.

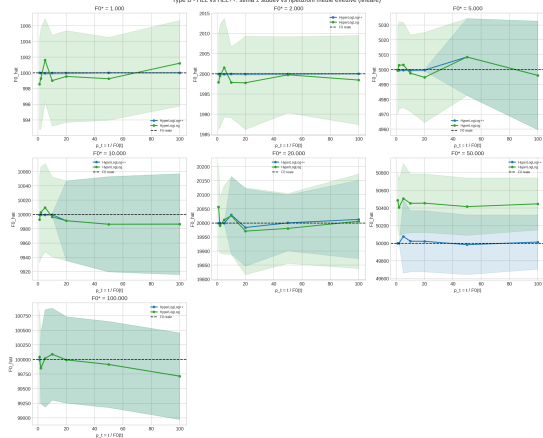
Type B - tutti gli algoritmi: stima $\pm$ stdev vs (ripetizioni medie effettive (log-log)



Scala logaritmica.

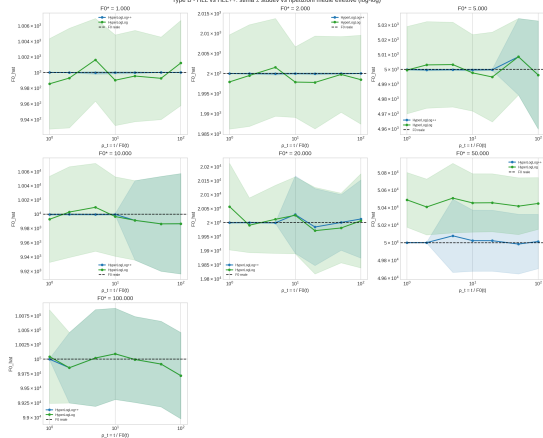
Caso B: confronto HyperLogLog e HyperLogLog++

Type B - HLL vs HLL++: stima $\pm$ stddev vs (ipotesi) medie effettive (lineare)



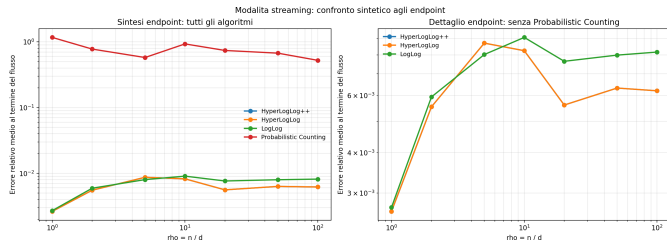
Scala lineare.

Type B - HLL vs HLL++: stima $\pm$ stddev vs (ipotesi) medie effettive (log-log)



Scala logaritmica.

L'errore relativo medio alla fine del flusso osservati sintetizza la qualità della stima nelle due viste già introdotte: in funzione di $F_0(t)$ e di $\rho_t = t/F_0(t)$.



Osservazioni

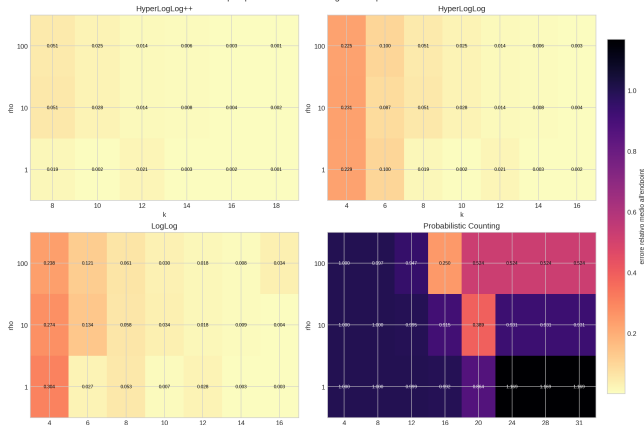
- sulla vista in $F_0(t)$, HLL e HLL++ sono quasi sovrapposti: errore medio circa 0.00618;
- sulla vista in ρ_t , HLL++ è il migliore: $3.95 \cdot 10^{-4}$, contro $2.08 \cdot 10^{-3}$ di HLL;
- LogLog è più sensibile alla vista sperimentale;
- Probabilistic Counting resta nettamente separato dagli altri;
- HLL++ è l'unico metodo che resta primo in entrambe le viste.

Risultati streaming: analisi del parametro di precisione

Configurazione

Per isolare l'effetto del parametro interno si fissano dataset, seed e hash (splitmix64, seed = 21041998) e si varia solo la precisione dell'algoritmo per $\rho \in \{1, 10, 100\}$.

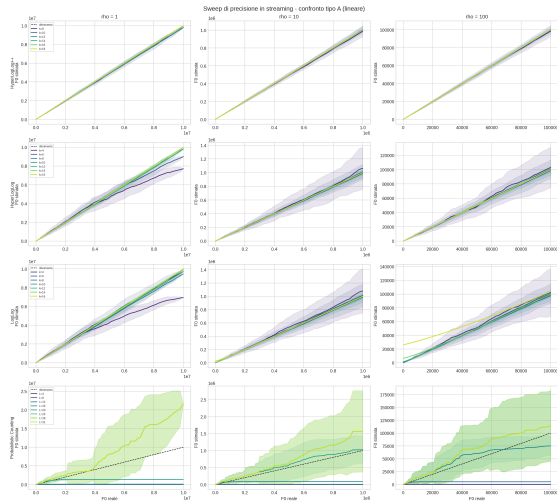
Sweep di precisione in streaming: heatmap dell'errore finale



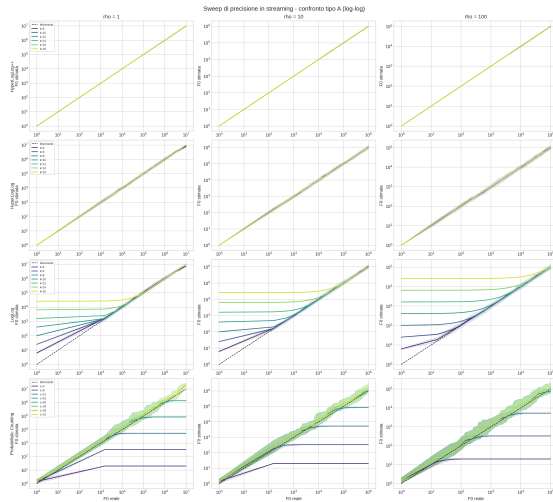
Miglior parametro osservato

- HLL++: $k = 18$ per tutti i valori di ρ ;
- HLL: $k = 16$ in media, ma con $\rho = 1$ basta $k = 10$;
- LogLog: $k = 14$ in media, con preferenza per $k = 16$ a $\rho = 10$;
- PC: $L = 20$ in media, con $L = 16$ quando $\rho = 100$;
- solo HLL++ migliora in modo davvero sistematico.

Parametro di precisione: confronto sui prefissi



Scala lineare.



Scala logaritmica.

Risultati sul merge distribuito

Esperimenti di merge HLL++: struttura

Procedura comune

Si costruiscono due sketch locali su partizioni distinte e uno sketch seriale sulla stessa unione dei dati; il merge viene poi confrontato con il riferimento seriale.

Controllo omogeneo

$$\rho \in \{1, 10, 100\}, \quad k \in \{10, 14, 18\}$$

Hash compatibile e stessa semantica sui due lati.

Strategia osservata: direct.

Quantità registrate

Per ogni confronto si salvano unione esatta, stima di merge, stima seriale ed errori assoluti e relativi rispetto al riferimento.

Nel caso `valid`, il controllo atteso è

$$\Delta_{abs} = 0, \quad \Delta_{rel} = 0.$$

Merge omogeneo: controllo del caso `valid`

Prima di studiare la degradazione data da un merge non ottimale bisogna verificare che, in condizioni pienamente compatibili, merge e processamento seriale coincidano davvero.

Configurazione

HyperLogLog++ con

$$\rho \in \{1, 10, 100\}, \quad k \in \{10, 14, 18\}$$

e hash compatibile sui due lati del merge.

Risultato osservato

$$\Delta_{abs} = 0, \quad \Delta_{rel} = 0$$

- per tutti i regimi ρ , il merge diretto riproduce il processamento seriale;
- non emerge alcuna differenza tra stima di merge e stima seriale;
- il caso `valid` diventa il riferimento per interpretare i mismatch successivi.

Esperimenti di merge HLL++: casi eterogenei

Caso invalid: hash mismatch

Precisione fissata a $k = 14$. Il controllo omogeneo locale viene confrontato con seed mismatch e family mismatch, anche invertendo i lati del merge.

Strategie: reject, unsafe_naive_merge.

Caso recoverable: precision mismatch

Hash fissata a xxhash64 con seed comune.

Si usano precisioni diverse, considerate in entrambi gli ordini di merge, mantenendo invariata la semantica dell'hash.

Strategie: reject, unsafe_naive_merge, reduce_then_merge.

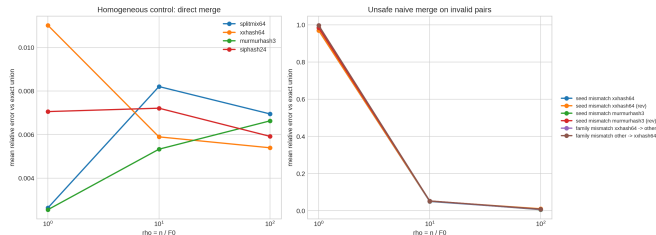
Riferimenti di confronto

reject registra esplicitamente l'incompatibilità; unsafe_naive_merge è il controllo negativo; nel caso recoverable, reduce_then_merge usa come riferimento omogeneo la precisione minima $\min(k_{left}, k_{right})$.

Hash mismatch: caso invalid

Configurazione

Precisione fissata a $k = 14$. Si osservano seed mismatch e family mismatch, con controllo omogeneo locale e scenari invertiti rev. Le strategie usate sono reject e unsafe_naive_merge.



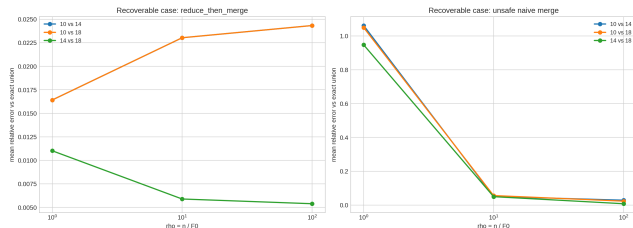
Osservazioni

- controllo omogeneo locale: errore medio = 0.006233;
- merge forzato nei casi invalid: errore medio tra 0.342374 e 0.352802;
- errore massimo osservato: 1.009634, nel regime $\rho = 1$;
- invertire i lati non ripara il problema semantico;
- operativamente il caso va rifiutato: reject.

Precision mismatch: caso recoverable

Configurazione

Hash xxhash64, seed comune e precisioni diverse tra i due sketch, considerate in entrambi gli ordini di merge. Si confrontano reject, unsafe_naive_merge e reduce_then_merge.



Strategie a confronto

Strategia	$\bar{\epsilon}_{merge}$
reject	—
unsafe_naive_merge	0.363882
reduce_then_merge	0.016651

La riduzione alla precisione minima comune riporta al merge sul riferimento seriale e sul corrispondente riferimento omogeneo. Il merge forzato raggiunge invece massimo 1.061378 e degrado medio 32.37.

Conclusioni

Conclusioni: cosa mostrano i risultati

Stima locale

- Nel dominio osservato, HLL++ è il metodo più stabile.
- HyperLogLog resta il metodo più vicino per comportamento, ma meno regolare.
- LogLog e soprattutto PC dipendono di più dal regime sperimentale e dal parametro scelto.

Merge distribuito

- il caso omogeneo coincide con il riferimento seriale;
- due funzioni di hash diverse rompono la semantica comune del merge;
- due parametri di precisione diversi restano recuperabili con normalizzazione.

Tesi centrale

Negli sketch count-distinct distribuiti, algoritmo, hash, seed e parametri fanno parte della semantica dello sketch: il merge è corretto solo entro questa semantica comune.

Conclusioni: contributi del lavoro

Software

Implementazione uniforme di più algoritmi count-distinct, con interfaccia comune per aggiornamento, stima, merge e reset.

Metodo

Framework riproducibile che separa dataset, hash, algoritmi, metriche e serializzazione e supporta streaming, merge omogeneo e merge eterogeneo.

Risultato principale

Distinzione operativa tra casi `valid`, `recoverable` e `invalid`, con evidenza sperimentale sulle condizioni del merge.

Contributo complessivo

Il lavoro non si limita a confrontare gli algoritmi di sketch ma esplicita quando il merge distribuito sia semanticamente corretto, quando vada rifiutato e quando richieda una trasformazione preliminare definita.

I risultati chiariscono bene il problema semantico di base, ma non esauriscono ancora la varietà dei flussi reali e delle topologie distribuite.

Dataset

La valutazione usa soprattutto un dataset sintetico controllato, utile per isolare i fenomeni ma non sufficiente da solo a rappresentare tutti i carichi reali.

Topologia

Lo studio del merge si concentra su casi pairwise, che chiariscono la semantica del problema ma non coprono catene o alberi di aggregazione più ampi.

Assi osservati

I casi eterogenei osservati sono due assi mirati (`hash mismatch`, `precision mismatch`) e il merge è sviluppato soprattutto su HLL++ come riferimento principale.

Dati reali

Portare il framework su dataset reali o su sorgenti di flusso come Apache Kafka, per osservare gli algoritmi fuori dal solo contesto sintetico.

Topologie e costi

Studiare topologie di merge più ampie e il punto ottimale in cui ridurre alla precisione minima, misurando anche costo di normalizzazione e merge.

Nuovi sketch

Estendere il framework ad algoritmi count-distinct più recenti (per esempio UltraLogLog [12]) e, più avanti, ad altre famiglie di sketch.

Stesso principio

Le estensioni future hanno senso soprattutto lungo assi di eterogeneità la cui interpretazione semantica sia chiara, come in questo lavoro.

- [1] S. Muthukrishnan, "Data streams: Algorithms and applications," Rutgers University, Tech. Rep., 2005.
- [2] Z. Bar-Yossef, T. S. Jayram, R. Kumar, D. Sivakumar, and L. Trevisan, "Counting distinct elements in a data stream," in *Randomization and Approximation Techniques in Computer Science (RANDOM 2002)*, ser. Lecture Notes in Computer Science, vol. 2483. Springer, 2002, pp. 1–10.
- [3] G. Cormode, "Data sketching," *Communications of the ACM*, 2017.
- [4] D. M. Kane, J. Nelson, and D. P. Woodruff, "An optimal algorithm for the distinct elements problem," in *Proceedings of the 29th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS 2010)*. ACM, 2010.
- [5] P. K. Agarwal, G. Cormode, Z. Huang, J. M. Phillips, Z. Wei, and K. Yi, "Mergeable summaries," in *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS 2012)*. ACM, 2012.
- [6] N. Alon, Y. Matias, and M. Szegedy, "The space complexity of approximating the frequency moments," *Journal of Computer and System Sciences*, vol. 58, no. 1, pp. 137–147, 2002.
- [7] P. Flajolet and G. N. Martin, "Probabilistic counting algorithms for data base applications," *Journal of Computer and System Sciences*, vol. 31, no. 2, 1985.
- [8] M. Durand and P. Flajolet, "Loglog counting of large cardinalities," in *Proceedings of the European Symposium on Algorithms (ESA 2003)*, 2003.

- [9] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, "Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm," in *Proceedings of the 2007 Conference on Analysis of Algorithms (AofA 07)*, 2007, pp. 127–146.
- [10] S. Heule, M. Nunkesser, and A. Hall, "Hyperloglog in practice: Algorithmic engineering of a state of the art cardinality estimation algorithm," in *Proceedings of the International Conference on Extending Database Technology (EDBT/ICDT '13)*. ACM, 2013.
- [11] O. Ertl, "New cardinality estimation algorithms for hyperloglog sketches," 2017, arXiv preprint. [Online]. Available: <https://oertl.github.io/hyperloglog-sketch-estimation-paper/paper/paper.pdf>
- [12] —, "Ultraloglog: A practical and more space-efficient alternative to hyperloglog for approximate distinct counting," 2024, VLDB 2024 / arXiv preprint. [Online]. Available: <https://arxiv.org/abs/2308.16862>

Grazie per l'attenzione.
Domande?